

License Plate Detection Using YOLO

ITAI 1378 – Midterm Project Blueprint
Tier 1 – Core Project

Team Members

Sandhya Chamakuri
Keichara Opara
Dulce Zuniga

Problem: License Plate Detection

Real-World Problem

- Manually locating license plates in large numbers of vehicle images can be time-consuming and inefficient.
- License plates can appear at different sizes, angles, distances, and lighting conditions, making consistent detection challenging.

Who Cares?

- Parking management systems
- Traffic monitoring systems
- Organizations that process vehicle images

Why It Matters

- Automatic detection can quickly locate license plates in vehicle images.
- It can provide a foundation for efficient vehicle monitoring and identification applications.

Solution: YOLO License Plate Detection

One-Sentence Solution:

Our system uses a trained YOLOv8n object detection model to automatically locate license plates in vehicle images and display bounding boxes with confidence scores.

Workflow:

Vehicle Image → YOLOv8n Model → License Plate Detection → Bounding Box + Confidence Score

Input

Vehicle image

↓

Processing

Trained YOLOv8n model

↓

Output

License plate bounding box + confidence score



Technical Approach

Computer Vision Technique

- Object Detection

Model / Architecture

- YOLOv8n (YOLO – You Only Look Once)
- Pretrained YOLOv8n model
- Trained for one class: license_plate

Framework & Environment

- Ultralytics YOLO
- Python
- Google Colab
- NVIDIA T4 GPU
- OpenCV

Why YOLO?

- Detects and locates license plates using bounding boxes.
- Fast and suitable for object detection.
- YOLOv8n is lightweight and practical for Google Colab.

Resources

- Free and open-source tools and libraries.

Data Plan

Data Source

- Kaggle – License Plate Dataset
- Dataset: ronakgohil/license-plate-dataset
- Downloaded from Kaggle using the KaggleHub library

Dataset Size

- Total images: **1,695**
- Training images: **1,526**
- Validation images: **169**
- Number of classes: **1 (license_plate)**

Labels & Format

- YOLO bounding-box annotation format
- License plate annotations were already provided with the dataset.

Data Preparation

- Verified image and label file counts.
- Inspected sample vehicle images and license plate annotations.
- Confirmed bounding boxes were correctly placed around license plates.
- Configured dataset paths for training and validation in Google Colab.

Success Metrics

Success Targets

- Detect license plates correctly in vehicle images.
- Achieve **Precision $\geq 90\%$** .
- Achieve **Recall $\geq 90\%$** .
- Achieve **mAP50 $\geq 90\%$** .
- Produce correctly positioned bounding boxes around license plates.
- Review multiple validation images for successful and unsuccessful detections.

Preliminary Results

- Precision: **0.994 (99.4%)**
- Recall: **0.991 (99.1%)**
- mAP50: **0.994 (99.4%)**
- mAP50-95: **0.882 (88.2%)**

Evaluation Method

- YOLO validation metrics
- Visual inspection of bounding boxes and confidence scores.

Milestone Plan

Blueprint

- Define the problem, project scope, technical approach, dataset, success metrics, and risks.

First Working Demo

- Run a pretrained YOLOv8n model successfully on sample images.

Make It Our Own

- Prepare and verify the license plate dataset.
- Train YOLOv8n on the license_plate class.

Improve & Measure

- Test the trained model on validation images.
- Evaluate Precision, Recall, mAP50, and mAP50-95.
- Review successful and unsuccessful detections.

Package & Present

- Organize the GitHub repository and documentation.
- Complete the presentation and prepare the project for submission.

Risks, Plan B & Compute Plan

Risk 1: Dataset and Annotation Issues

- Some license plates may be small, unclear, angled, or difficult to detect.
- **Plan B:** Review images and annotations and correct or remove problematic samples if needed.

Risk 2: Google Colab / GPU Limitations

- Runtime disconnections or limited GPU availability could interrupt training.
- **Plan B:** Use the lightweight YOLOv8n model and save the trained model, results, and notebook backups to Google Drive.

Compute Plan

- Google Colab with NVIDIA T4 GPU.
- YOLOv8n for efficient training and detection.
- Python and Ultralytics YOLO.
- We used free and open-source tools and resources for this project.